

AI Quality Engineering — Class 2

Student Handout: AI Model as API, Data Drift and Deployment Testing

Overview

In Class 1, we treated the AI model as an object to study in isolation — measuring accuracy, generalisation, and robustness. In Class 2, we shift perspective entirely.

A model is not a product. A deployed service is a product.

Real users never touch a model le. They call an API. They expect responses. They expect consistency. They expect reliability — not just accuracy.

This class covers three layers of system-level AI quality:

Layer	Focus	What We Test
Interface Layer	AI model wrapped in a REST API	HTTP behaviour, input validation, response structure, latency
Data Layer	Data quality and distribution over time	Schema validity, statistical drift, distribution monitoring
Deployment Layer	Releasing model updates safely	Versioning, tra c routing, A/B testing, rollback

PART A: THE AI MODEL AS AN API

1. Why System Testing is Different from Model Testing

After evaluating a model in Class 1 — checking accuracy, generalisation gap, robustness — you might think the model is ready for the world. But model quality and system quality are two different things.

Consider this scenario:

- Your model achieves 92% accuracy on a clean test set.
- A user uploads a corrupted image le. The server crashes.
- A mobile app sends a text le by mistake. The system throws an unhandled exception.
- The response format changes slightly in one deployment. The frontend breaks silently.

From the user's perspective, the system is unreliable — even though the model is accurate.

A model can be intelligent and still be unreliable as a deployed service.

This is why AI quality engineering expands the scope beyond model validation into **system validation**.

2. What is a REST API?

When an AI model is deployed as a service, it is typically exposed via a REST API — a set of endpoints that accept HTTP requests and return HTTP responses.

A typical AI inference API exposes endpoints such as:

Endpoint	Method	Purpose
/health	GET	Is the server alive and running?
/info	GET	What model is loaded? What are its metadata?
/predict	POST	Submit input and receive a prediction

A client — a mobile app, web frontend, or another service — sends a request. The server processes it, runs the model, and returns a structured JSON response.

Example Prediction Response:

```
1  {  
2  
3    "prediction": "class_A",  
4    "confidence": 0.91,  
5    "model_version": "v2.0",  
6    "latency_ms": 34.5  
7  }
```

Python

This structure must be:

- **Consistent** — same fields every time
- **Typed correctly** — confidence must be a float, not a string
- **Complete** — no missing fields
- **Bounded** — confidence must be between 0.0 and 1.0

3. HTTP Status Codes: The Language of Reliability

HTTP status codes are how systems communicate the outcome of a request. In AI APIs, they are a critical quality signal.

Status Code	Category	Meaning in AI APIs
200 OK	Success	Request processed, prediction returned
400 Bad Request	Client error	Invalid input (wrong type, corrupt data, too small)
422 Unprocessable Entity	Client error	Required field is missing from the request
429 Too Many Requests	Rate limit	Client has exceeded allowed request rate

500 Internal Server Error	Server error	Model crashed or unexpected server-side failure
---------------------------	--------------	---

Why Status Codes Matter for Testing:

- If an API returns 200 OK even when the model fails internally, monitoring systems cannot detect errors.
- If the API crashes with a 500 on corrupted input instead of returning a clean 400, the client cannot handle the error gracefully.
- If required elds are missing and the server returns 500 instead of 422, the error is misattributed.

Testing AI APIs means verifying that correct status codes are returned for every class of input — not just the happy path.

4. Input Validation: Testing Edge Cases

Every real-world API must handle unexpected inputs without crashing. For AI models speci cally, the types of bad inputs are well-defined:

Input Problem	Expected Response	What It Tests
Wrong le type (e.g., .txt instead of .jpg)	400 Bad Request	File type check
Corrupt image data (random bytes)	400 Bad Request	Image decodability check
Empty le (0 bytes)	400 Bad Request	Empty payload check
Image too small (e.g., 8×8 pixels)	400 Bad Request	Minimum dimension check
Missing required eld in the request	422 Unprocessable Entity	Required parameter check
Valid image, normal conditions	200 OK	Happy path

Testing all these cases explicitly is called **negative testing** — deliberately sending inputs you know are invalid to verify graceful failure.

A system that fails gracefully on bad input is **fault-tolerant**. A system that crashes is **brittle**.

5. JSON Schema Validation

Even when a request succeeds (HTTP 200), the response content must be validated. This is where **JSON schema validation** comes in.

A JSON schema defines the contract of a response:

- Which fields must be present
- What type each field must have
- What value ranges are valid

Example Schema for a Prediction Response:

Python

```

1  {
2
3    "prediction": "string (one of the valid class names)",
4    "confidence": "float between 0.0 and 1.0",
5    "model_version": "string matching version format",
6    "latency_ms": "float > 0"
7  }

```

If any part of this schema is violated — even if the HTTP status is 200 — the test should fail.

Why this matters: A backend upgrade might rename a field. A new model might add an extra field. A bug might change a float to a string. None of these trigger a 500 error, but all of them can break downstream consumers.

Consistency in structure is as important as correctness in prediction.

6. Latency and Performance Testing

Even if a model predicts correctly and the API responds with the right structure, a slow API degrades user experience.

Key Performance Metrics for AI APIs:

Metric	Definition	Why It Matters
Latency	Time from request to response (ms)	Determines user-perceived responsiveness
Throughput	Requests processed per second	Determines scalability
P95 / P99 Latency	95th or 99th percentile of latency	Reveals worst-case behaviour, not just average
Timeout Rate	% of requests that exceed a time threshold	Indicates system under stress

Why average latency is not enough:

A system with a mean latency of 50ms might have a P99 of 2000ms — meaning 1 in 100 requests takes 2 seconds. In a real-time system, those slow requests matter.

Latency testing involves:

- Sending N requests and measuring end-to-end time per request
- Computing mean, median, min, max, and standard deviation
- Comparing across model versions
- Verifying the system meets defined SLA (Service Level Agreement) thresholds

A slow correct system feels incorrect to users. Performance is part of quality.

7. The System Testing Mindset

When you shift from model testing to system testing, the questions change:

Model Testing Questions	System Testing Questions
Is accuracy high?	Is the service reliable?
Does the model generalise?	Are errors handled gracefully?
Is the model robust to noise?	Is the interface consistent?
Is the model fast enough?	Does it meet latency SLAs?

The model is one component. The API is the contract between your AI and the world. If that contract is unreliable, the entire system loses trust

PART B: DATA LAYER — DISTRIBUTION AND DRIFT TESTING

1. The Silent Failure Problem

In Class 1, we tested the model against adversarial inputs and noise — deliberate, controllable perturbations. But in production, the more common threat is subtle and unintentional.

Models fail silently when data changes.

When you train a model, you implicitly assume that the data it encounters in production will resemble the training data. This assumption is not written in code. It is not validated automatically. It is simply assumed.

And when that assumption is violated — as it often is — the model degrades without any crash, no exception, no error log. Just quietly lower accuracy.

This is the silent failure problem.

2. What is Data Drift?

Data drift is any change in the statistical properties of the input data over time, after a model has been deployed.

Common real-world causes:

Cause	Example
Sensor change	Camera hardware upgrade changes image brightness characteristics
Environmental change	Model trained on summer data deployed in winter
Behavioral change	User patterns evolve over months
System change	Upstream service changes preprocessing pipeline

In all these cases, the model weights did not change. The API did not change. But the input distribution changed — and model performance degrades as a result.

3. Types of Drift

Understanding exactly what changed is essential for choosing the right response.

Covariate Shift

The distribution of input features changes, but the relationship between features and labels remains the same.

Example: A model trained on daytime road images is deployed and receives nighttime images. The images are darker, but the labels are still correct — a stop sign is still a stop sign. The input statistics changed, but the meaning did not.

Impact: Model accuracy drops because the input looks different from training data.

Response: Retrain on data that includes the new distribution.

Concept Drift

The relationship between input features and labels changes over time.

Example: A fraud detection model learns patterns of fraudulent transactions in 2022. By 2025, fraud strategies have evolved. The same behavioral pattern that indicated fraud before now looks like normal activity. The input statistics may be similar, but what those inputs mean has changed.

Impact: Model accuracy drops because the learned mapping is no longer valid.

Response: Requires new labelled data and full retraining, not just ne-tuning.

Type	What Changed	What Stayed the Same
Covariate Shift	Input distribution $P(X)$	Label relationship $P(Y X)$
Concept Drift	Label relationship $P(Y X)$	Input distribution $P(X)$

4. Schema Validation vs. Distribution Validation

There are two levels of data quality checks:

Schema Validation (Structural)

Checks that the data has the correct format, types, and elds:

- Are all expected columns/ fields present?
- Are data types correct (e.g., image tensor vs. null)?
- Are there missing values?
- Is the tensor shape correct?

Schema validation is necessary but not sufficient. A perfectly structured input can be statistically very different from what the model expects.

Distribution Validation (Statistical)

Checks that the statistical properties of the data are consistent with the training distribution:

- Has the mean or variance of pixel values shifted?
- Has the class balance changed?
- Has the distribution of input features drifted?

You need both. Schema validation protects you from structural corruption. Distribution validation protects you from statistical degradation.

5. Measuring Drift: KL Divergence

To quantify how much a distribution has changed, we need a metric. One of the most widely used is KL Divergence (Kullback-Leibler Divergence). Intuition: KL Divergence measures how different one probability distribution is from another reference distribution.

$$1 \quad KL(P \parallel Q) = \sum P(x) \times \log(P(x) / Q(x))$$

Python

Where:

- P = reference distribution (training data)
- Q = current distribution (production data)

KL Value	Interpretation
Near 0	Distributions are very similar — low drift
0.1 – 0.5	Moderate drift — worth monitoring
> 0.5	High drift — likely performance degradation

Key insight: KL divergence is computed on feature distributions, not on labels. You can detect drift proactively — before you see accuracy drop — because you are monitoring the input, not waiting for the output to fail.

6. Drift Detection System

A drift detection system is an automated layer that monitors incoming data and raises alerts when thresholds are exceeded.

A typical Drift Detector has Two Checks:

Accuracy Drift Check

- Compare current model accuracy on recent data to the baseline accuracy
- If the drop exceeds a threshold (e.g., 5%), raise an alert

Distribution Drift Check

- Compute KL divergence between recent input distribution and training distribution
- If KL divergence exceeds a threshold (e.g., 0.5), raise an alert

When alerts are triggered, the system recommends action:

Alert Type	Recommendation
------------	----------------

Accuracy drift only	Investigate specific class failures; may be sampling bias
Distribution drift only	Monitor accuracy closely; retrain may be needed soon
Both triggered	Initiate retraining pipeline immediately

7. Monitoring Strategies

Batch Monitoring

- Aggregate production data over a time window (e.g., daily)
- Compute statistics and compare against training distribution
- Suitable for systems where slight delay in detection is acceptable

Real-Time Monitoring

- Compute rolling statistics on a stream of incoming data
- Detect distribution shift immediately as it happens
- Required for safety-critical or high-throughput systems

In safety-critical systems, the delay in batch monitoring may be unacceptable. If a camera update shifts all inputs overnight, real-time monitoring detects it immediately; batch monitoring may take 24 hours.

8. The Production Drift Workflow

```

1  Train model on dataset D
2      → Deploy with monitoring
3      → Monitor input distribution (KL divergence)
4      → Monitor accuracy on labeled subset
5      → Drift threshold exceeded?
6          → Yes: Collect fresh data → Retrain → Validate → Redeploy
7          → No: Continue monitoring

```

Python

Key principle: In production AI, the most dangerous failures are silent failures. There is no stack trace for drift. There is no crash log for distribution mismatch. There is only slow, invisible degradation.

Distribution testing is not an optional quality step. It is a core production responsibility.

PART C: DEPLOYMENT LAYER — VERSIONING AND TRAFFIC TESTING

1. Why Model Updates Are Risky

In research, when you train a better model, you replace the old one. Simple.

In production, doing that blindly is dangerous.

Why? Because production systems are complex:

- **Latency constraints:** A more accurate model might be slower.

- **Unexpected edge cases:** The new model may have been tested on a clean validation set but never seen certain production inputs.
- **Hardware differences:** The model behaves differently in a production environment vs. a development environment.
- **System integrations:** Downstream consumers may depend on specific output formats or confidence distributions.

A higher offline accuracy metric does not guarantee a better production model.

This is the core insight driving everything in this section.

2. Model Versioning and the Model Registry

Before you can manage model deployments, you need to manage model identity. Every model in production must be versioned and tracked.

What a Model Version Captures:

Attribute	Why It Matters
Training dataset version	Reproducibility — what data produced this model?
Hyperparameters	Can you retrain the exact same model if needed?
Code version	Which training code was used?
Deployment timestamp	When was this version live?
Performance metrics	What were the validation results?

A **model registry** is a central system where models are stored with all this metadata. Instead of using "the latest model," you reference "version 1.3.2, trained on dataset v5, deployed 2025-09-01."

Why Versioning Matters:

- If something breaks in production, you can roll back to the previous version.
- If you cannot reproduce a model, you cannot diagnose why it behaved differently.
- If you do not track which version is live, you cannot correlate user complaints to a specific model.

A model you cannot trace is a model you cannot trust.

3. Deployment Strategies

There are several strategies for safely releasing model updates, each with different risk profiles.

Blue-Green Deployment

Two identical environments: one "live" (Blue = current production), one "staging" (Green = new version).

```
1 Blue (v1) = 100% of production traffic
2 Green (v2) = Staged, fully tested
3 When v2 passes validation:
4
```

Python

- 4 → Switch ALL traffic **from** Blue to Green instantly
- 5 → If issues arise → switch back to Blue immediately

Advantages: Zero downtime. Instant rollback.

Disadvantages: Requires double the infrastructure. Switch affects all users at once.

Canary Deployment

Release the new model to a small percentage of traffic first.

- 1 v1 = 95% of traffic (stable, trusted)
- 2 v2 = 5% of traffic (new version, under observation)
- 3 Monitor metrics:
- 4 → If v2 performs well: 5% → 25% → 50% → 100%
- 5 → If v2 degrades: Roll back to 0% immediately

JavaScript

Advantages: Risk is contained. Real-traffic validation. Small blast radius on failure.

Disadvantages: Slightly more complex routing logic.

Shadow Deployment

The new model receives real production traffic, but its predictions are not returned to users.

- 1 User request arrives
- 2 → v1 processes it → returns prediction to user
- 3 → v2 processes it simultaneously → logs result (**not** returned to user)
- 4 Offline: compare v1 **and** v2 outputs

Python

Advantages: Zero risk to users. Real production data validation.

Disadvantages: No real user impact to measure quality. Requires extra compute.

A/B Testing

Traffic is split between two models, and both predictions are used in real interactions.

- 1 Group A: 50% of users → served by v1
- 2 Group B: 50% of users → served by v2
- 3 Measure: accuracy, latency, business metrics, user satisfaction
- 4 → Declare a winner using statistical significance testing
- 5 → Deploy the winner to 100% of traffic

Python

Advantages: Real-world performance data. Statistically rigorous comparison.

Disadvantages: Both models must be acceptable quality; you cannot A/B test a broken model against a good one.

4. Comparison of Deployment Strategies

Strategy	User Impact	Risk Level	Rollback Speed	Best For
Blue Green	None during switch	Medium	Instant	Minor updates, fully tested versions
Canary	Partial (small % of users)	Low	Fast	Major updates, unknown edge cases
Shadow	None	None	N/A	High-stakes validation, zero tolerance for risk
A/B Testing	Full (all users)	Medium	Medium	Feature comparison, business decisions

5. What to Monitor During Deployment

A better online metric does not guarantee a better production model. This is why monitoring during deployment is not optional.

When comparing model versions in production, you do not monitor accuracy alone.

The full Monitoring Checklist:

Metric	Why Monitor It	Alert Condition
Accuracy / Error Rate	Primary quality signal	Drop > threshold vs. baseline
Latency (mean, P95, P99)	Performance regression	New version > 20% slower
Throughput	Scalability	Requests per second drops
Agreement Rate	Prediction stability	v1 and v2 disagree > 30% of the time
Confidence Distribution	Output quality	Confidence systematically lower in new version
Drift Metrics (KL)	Input distribution	Simultaneous drift confounds version comparison

6. Agreement Rate: A Key Canary Metric

When two model versions are running simultaneously, **agreement rate** measures what percentage of the time they produce the same prediction on the same input.

$$1 \quad \text{Agreement Rate} = (\text{Number of identical predictions} / \text{Total predictions}) \times 100\% \quad \text{Python}$$

Interpreting agreement rate:

Agreement Rate	Interpretation
----------------	----------------

> 90%	Models are largely equivalent; v2 is safe to promote
70–90%	Moderate disagreement; investigate specific inputs where they differ
< 70%	Significant behavioral change; stop the canary and investigate

High disagreement does not mean one model is wrong — it means the two models are making systematically different decisions. That requires understanding, not just a metric.

7. When to Stop a Canary / Roll Back

Stop the Canary If:

- v2 latency exceeds $2 \times$ v1 latency
- v2 error rate exceeds 5%
- Agreement rate drops below 70%
- Confidence scores are systematically lower on v2
- Any production alert is triggered that correlates with v2 deployment

Roll back v2 completely if:

- Accuracy drops below the production threshold
- Error rate spikes suddenly
- Latency consistently exceeds SLA
- User-reported issues increase following the deployment

The key insight: Canary deployment is designed to contain the blast radius of a bad deployment. If you deployed to 5% of traffic and detected a problem, you affected 5% of users — not 100%.

8. Throughput and Rate Limiting

Throughput measures how many requests an API can process per unit of time.

Throughput = Total Requests Processed / Total Time (seconds)

Why throughput matters: A model that handles 10 requests per second but receives 100 per second will queue requests, increasing latency for all users. Under sustained overload, the system may crash.

Rate limiting is the mechanism by which an API controls the maximum number of requests a client can send in a time window.

Python

1 Example: 100 requests per minute per client → If exceeded: HTTP 429 Too Many Request

◀

▶

Rate Limiting Serves Two Purposes:

1. **Protection from overload** — prevents any single client from saturating the system .
2. **Protection from abuse** — reduces the risk of denial-of-service attacks

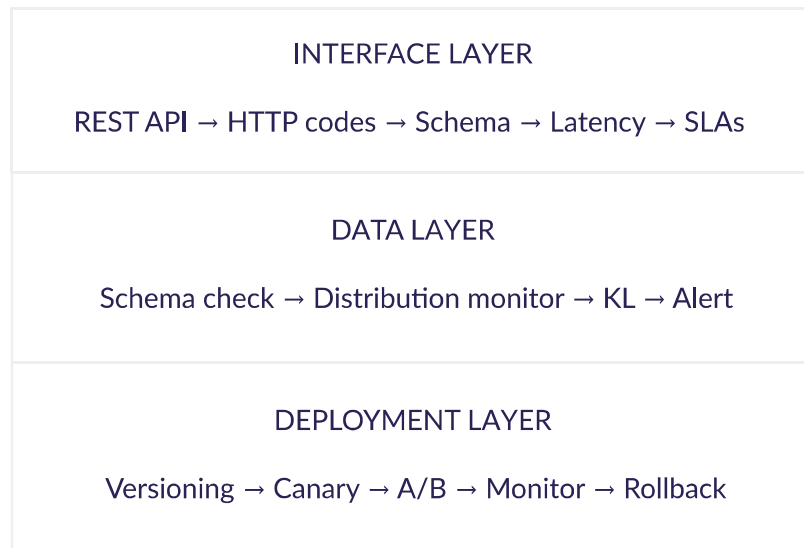
In production AI APIs, rate limiting must be tested explicitly:

- Does the API correctly return 429 when limits are exceeded?

- Does the API recover after the rate limit window resets?
- Are rate limits applied per client, per IP, or globally?

9. Connecting the Three Layers

In this class, we have built a complete system-level view of AI quality:



These layers interact:

- A canary deployment that sees rising KL divergence may be experiencing distribution shift rather than a v2 problem – the layers must be analysed together.
- An API that handles errors gracefully will not mask data-layer problems, making drift easier to detect.
- A well-versioned model registry makes rollback trivial, because you always know exactly what is running.

Production AI is not a one-time deployment. It is a continuous, monitored, versioned system.

PART D: KEY CONCEPTS SUMMARY

API Testing Framework

Test Category	What Is Tested	Expected Behavior
Happy path	Valid input	HTTP 200, correct prediction
File type validation	Wrong extension	HTTP 400
Corrupt input	Undecodable file	HTTP 400
Empty input	Zero bytes	HTTP 400
Dimension check	Below minimum size	HTTP 400
Missing parameter	Required eld absent	HTTP 422
Rate limit	Too many requests	HTTP 429

Server error	Internal failure	HTTP 500
--------------	------------------	----------

Drift Testing Framework

Test Type	Technique	Metric	Alert Threshold
Structural validation	Schema check	Pass / Fail	Any failure
Distribution monitoring	KL Divergence	Scalar value	> 0.5
Accuracy monitoring	Accuracy on labeled samples	% accuracy	Drop > 5% from baseline

Deployment Strategy Selection Guide

Scenario	Recommended Strategy
Minor update, fully validated o ine	Schema check
Major model change, uncertain edge cases	KL Divergence
High-stakes system, zero tolerance for user impact	Accuracy on labeled samples
Choosing between two competing models	A/B Testing
Emergency rollback needed	Blue-Green (instant switch back)

PART E: KEY FORMULAS & DEFINITIONS

Formula / Concept	Definition
$KL(P \parallel Q) = \sum P(x) \log(P(x)/Q(x))$	Measures how distribution Q differs from reference P
Agreement Rate = (Matching predictions / Total) × 100%	Measures prediction consistency between two model versions
Throughput = Requests / Time (seconds)	Measures API request processing rate
Robustness Score = (Robust Acc / Clean Acc) × 100%	From Class 1 — repeated for reference
Generalization Gap = Train Acc – Val Acc	From Class 1 — repeated for reference

PART F: GLOSSARY

Term	Definition
------	------------

REST API	A web service that exposes endpoints for clients to send requests and receive responses
HTTP Status Code	Numeric code indicating the outcome of an HTTP request (200, 400, 422, 500, etc.)
Input Validation	Checking that incoming data meets format, type, and value requirements before processing
JSON Schema	A formal specification of the structure and types required in a JSON response
Latency	Time between a request being sent and a response being received
Throughput	Number of requests processed per unit of time
Rate Limiting	Restricting the number of requests a client can make in a time window
Data Drift	Any change in the statistical properties of input data after model deployment
Covariate Shift	When the input distribution $P(X)$ changes but the label relationship $P(Y X)$ remains the same
Concept Drift	When the relationship between inputs and labels $P(Y X)$ changes over time
KL Divergence	A measure of how different one probability distribution is from a reference distribution
Schema Validation	Checking that data has the correct structure, types, and fields
Distribution Validation	Checking that data has consistent statistical properties relative to training distribution
Silent Failure	A system failure that produces no crash or error log — only degraded output quality
Model Registry	A central system for storing, versioning, and tracking deployed models
Model Versioning	Assigning unique, traceable identifiers to model artifacts with full metadata
Blue-Green Deployment	Maintaining two environments and switching all traffic between them instantly
Canary Deployment	Gradually routing a small % of traffic to a new version before full rollout
Shadow Deployment	Running a new model in parallel with production but not returning its predictions to users
A/B Testing	Splitting traffic between two versions and comparing real-world performance
Agreement Rate	The % of requests where two model versions produce the same prediction

SLA (Service Level Agreement)	A performance commitment — e.g., 99% of requests respond within 200ms
Rollback	Reverting production to a previous model version when problems are detected
P95 / P99 Latency	The latency value below which 95% or 99% of requests fall
Batch Monitoring	Computing distribution statistics over an aggregated time window
Real-Time Monitoring	Continuously computing rolling statistics on streaming data